Python 基础知识专题讲义与配套练习

集合（set）、字典（dict）与购物车基础应用

学习目标：只掌握最基础、最典型、可直接模仿的知识与操作。
题目严格控制为入门级：10 道单选 + 5 道多选 + 5 道填空 + 4 道基础解答，共 24 道。

生成日期：2026 年 8 月 19 日

使用说明

本资料根据上传的 5 张手写笔记（Memo No. 198–202）整理。为保证可复制、可检索和可打印，正文**不嵌入原始照片**，而是把其中的文字、代码和表格重新编码为 LaTeX。图片中的核心主题依次是：集合的定义与常用操作、集合运算与集合推导式、字典的定义与访问方式、字典常用操作、五类常见容器对比，以及一个使用字典和 `match/case` 实现的购物车案例。

本资料区分三种内容：**原图转录**、**必要纠错**、**规范教学表述**。凡原图存在明显语法或概念问题，先说明原写法，再给出修正；凡手写内容略模糊，则明确标注，不把猜测冒充原文。

目录

使用说明 **1**

1 图片内容重新编码 **2**

1.1 Memo No. 198：集合的基本操作 2
1.2 Memo No. 199：集合解包、集合运算总结与字典定义 3
1.3 Memo No. 200：字典常用操作与容器对比 3
1.4 Memo No. 201：购物车功能与主循环雏形 4
1.5 Memo No. 202：Case 1 与 Case 2 的手写代码 5

2 原图问题与必要纠错 **6**

3 知识点清单 **7**

4 核心概念详解 **7**

4.1 集合 set：无序、去重、可修改 7
4.2 集合的四个修改方法 8
4.3 交集、并集、差集 8
4.4 集合推导式 9
4.5 星号 *：把可迭代对象拆成多个函数参数 10
4.6 字典 dict：通过 key 找 value 10
4.7 字典的增、改、删、查 11
4.8 五类常见容器的最低限度对比 12
4.9 input 与 match/case：购物车为什么匹配字符串 12

5 最基础示例 **12**

5.1 示例 1：去重 12
5.2 示例 2：求共同元素 12
5.3 示例 3：用字典保存成绩 13

5.4 示例 4：最小购物车数据结构 13

6 经纠错后的完整购物车基础程序 13

7 知识点—题目对应表 16

8 单项选择题（10 道） 16

9 多项选择题（5 道） 18

10 填空题（5 道） 19

11 基础解答题（4 道） 19

12 答案速查表 21

13 单项选择题详细解析 21

14 多项选择题详细解析 24

15 填空题详细解析 26

16 解答题详细解析 27

17 高频易错点总结 29

18 基础解题步骤总结 30

19 参考资料与核验来源 30

20 质量检查记录 31

1 图片内容重新编码

1.1 Memo No. 198：集合的基本操作

原图标题为“Python”“三、数据容器”“四、集合”。主要内容如下。

1. 基本操作

1. **介绍：**集合是一种“无序、不重复、可修改”的数据容器。
2. **定义：**原图给出一个非空集合字面量示例，形式为

```
s1 = {"a", "b", "x"}
```

其中个别手写字符受笔迹影响，但可以确认考点是“用花括号写非空集合”。原图同时给出空集合：

```
s2 = set()
```

3. 原图红字强调：**空集合不能写成 {}；{} 表示空字典。由于集合没有位置索引，因此不能像列表那样用下标查询元素。**
4. **常见操作：**

操作	原图含义	原图示例（规范化排版）
`add(...)`	添加元素到集合	`s1.add('t')`
`remove(...)`	移除集合中指定元素	`s1.remove('t')`
`pop()`	删除集合中的一个元素并返回	`e = s1.pop()`
`clear()`	清空集合	`s1.clear()`
`difference()`	求两个集合的差集	`s1.difference(s2)`
`union()`	求两个集合的并集	`s1.union(s2)`
`intersection()`	求两个集合的交集	`s1.intersection(s2)`

原图还写出：对于差集，不同集合放在前后位置，结果意义不同；集合的特点为“无序、不重复、可修改”。运算符等价形式写为：

$$S_1 \cap S_2 \leftrightarrow \texttt{s1 \& s2}, \qquad S_1 \setminus S_2 \leftrightarrow \texttt{s1 - s2}, \qquad S_1 \cup S_2 \leftrightarrow \texttt{s1 | s2}.$$

这里左侧是数学集合记号，右侧是 Python 写法。

原图最后给出集合推导式的一般形状：

```
{表达式 for 变量 in 可迭代对象 if 条件}
```

1.2 Memo No. 199：集合解包、集合运算总结与字典定义

2. 集合的“解包”

原图示例：

```
a = {10, 20, 30}
print(*a)
```

并把它近似解释为把集合中的元素作为多个位置参数传给 `print()`。

3. 集合运算符小结

原图总结：`&` 表示交集，`|` 表示并集，`-` 表示差集。集合推导式再次写为：

```
变量名称 = {表达式 for 变量 in 列表 if 条件}
```

4. 字典

原图定义：Python 中的字典 `dict` 存储“键值对（key:value）”，可以根据键 `key` 找到对应的值 `value`。

原图举例把三位同学的成绩保存到字典中，手写姓名中第三个姓名略模糊；可辨认的结构为：

```
dict1 = {
    "王林": 670,
    "李晓": 582,
    "松木": 551
}
```

其中第三个姓名按可见笔迹转录为“松木”，但不把姓名本身作为后续考点。

原图总结字典特点：键值对存储；键不能重复；字典可修改。基本语法写作：

```
字典名称 = {
    key: value,
    key: value,
    key: value
}
```

并给出“根据 key 获取 value”：

```
值 = 字典名称[key]
```

原图最后提醒：`value` 可以是多种类型，而 `key` 不能使用不合适的可变对象。

1.3 Memo No. 200：字典常用操作与容器对比

原图红字说明：若字典里出现重复的 key，后面的值会覆盖前面的值；字典不能用普通位置下标“第 0 个、第 1 个”来取值，而应根据 key 取 value。

5. 字典常见操作

类型	操作	含义
添加	字典名称 `[key] = value`	添加新的 key-value 键值对
删除	字典名称`.pop(key)`	删除指定 key，并返回对应 value
删除	`del` 字典名称 `[key]`	删除指定键值对
修改	字典名称 `[key] = value`	key 已存在时，更新对应 value
查询	字典名称 `[key]`	根据 key 获取 value
查询	字典名称`.get(key)`	根据 key 获取 value
查询	字典名称`.keys()`	获取所有 key 的视图
查询	字典名称`.values()`	获取所有 value 的视图
查询	字典名称`.items()`	获取所有 key-value 对的视图

6. 原图中的五类容器对比

特性	字符串	列表	元组	集合	字典
有序性	有序	有序	有序	无序	有序（见纠错说明）
重复元素	允许	允许	允许	不允许	key 不重复，value 可重复
可变性	不可变	可变	不可变	可变	可变
位置索引	支持	支持	支持	不支持	不支持位置索引
切片	支持	支持	支持	不支持	不支持
典型用途	文本	有序可修改数据	固定记录	去重/集合运算	键值映射

7. 购物车案例需求

原图提出“开发一个购物车管理系统，系统使用字典结构存储商品”，功能包括添加、修改、删除、查询和退出。

1.4 Memo No. 201：购物车功能与主循环雏形

原图的五项功能可以重新编码为：

1. **添加购物车：**录入商品名称、价格、数量，并保存商品信息。
2. **修改购物车：**输入要修改的商品名称，再输入新的价格和数量，修改该商品信息。
3. **删除购物车：**输入要删除的商品名称，根据名称删除商品。

4. **查询购物车：**显示购物车中的商品，格式类似“商品名称：xxx，商品价格：xxx，商品数量：xxx”。
5. **退出购物车**。

原图主循环雏形：

```
shopping_cart = {}
print("欢迎使用购物车系统!")
menu = """
##### 购物车系统 #####
# 1. 添加购物车      #
# 2. 修改购物车      #
# 3. 删除购物车      #
# 4. 查询购物车      #
# 5. 退出购物车      #
"""

while True:
    print(menu)
    choice = input("请输入您的操作(1-5):")
    match choice:
        ...
```

1.5 Memo No. 202：Case 1 与 Case 2 的手写代码

原图尝试写出“添加”和“修改”的分支。按手写内容忠实转录其意图如下（保留问题，不把它当作可运行代码）：

```
Case1:
    print("请添加商品到购物车中!")
    goods_name = input("请输入所添加的商品名称:")
    goods_price = input("请输入所添加的商品价格:")
    goods_num = input("请输入所添加的商品数目:")
    shopping_cart[goods_name] = {
        "商品价格": goods_price,
        "商品数目": goods_num
    }

Case2:
    if goods_name in shopping_cart:
        goods_name = input("请输入所修改的商品名称:")
        goods_name = input("请输入所修改的商品价格:")
        good_num = input("请输入所修改的商品数目:")
    else:
        print("无此商品,请重新输入!")

# 后续省略
```

2 原图问题与必要纠错

序号	原图写法/现象	问题	规范修正
1	`{}` 被提醒不能作为空集合	这一提醒是正确的	空集合必须写 `set()`；`{}` 是空字典。
2	把 `print(*a)` 近似写成 `print(10,20,30)`	集合是无序容器，输出顺序不应被保证	只能说“`*a` 把当前迭代到的元素拆成多个参数”；不要依赖其输出次序。
3	`set.pop()` 被理解为“随机删除”	“随机”容易让初学者误以为有概率保证	官方表述更接近“删除并返回任意一个元素”；程序不应依赖具体删掉哪个。
4	字典 key 被概括为“不可变”	“不可变”是入门近似，但严格条件是“可哈希 (hashable)”	字符串、数字、只含可哈希元素的元组通常可作 key；列表、字典、普通 set 不可作 key。
5	字典“有序”未说明版本语境	现代 Python 的 dict 保留插入顺序；老版本教材可能表述不同	Python 3.7 起语言规范保证字典按插入顺序保留条目；但访问仍是按 key，而非把字典当列表用位置下标。
6	`Case1:`、`Case2:`	Python 语法错误；关键字必须小写且 case 后要写模式	因 `choice = input(...)` 返回字符串，应写 `case "1":`、`case "2":`。
7	Case 2 先使用 `goods_name` 再输入要修改的名称	如果之前没有定义该变量会报错；逻辑顺序也反了	先 `goods_name = input(...)`，再判断 `if goods_name in shopping_cart:`。
8	第二次输入“商品价格”仍赋给 `goods_name`	覆盖了商品名变量，变量含义错乱	应赋给 `goods_price`。
9	`good_num` 与前文 `goods_num` 不一致	变量名不统一，容易导致后续取不到值	统一使用 `goods_num`。
10	价格、数量直接使用 `input()` 的结果	`input()` 返回字符串；若要做数值运算需转换	价格可用 `float(input(...))`，数量可用 `int(input(...))`。
11	修改分支只输入新值但未回写字典	用户输入后购物车数据不会真的改变	在校验存在后执行 `shopping_cart[name] = {...}` 或逐字段更新。
12	缺少 `case _:` 与退出分支实现	非法输入没有提示，`while True` 也无法退出	增加 `case "5": break` 和最后的 `case _:`。

核验结论： Python 官方文档明确说明，set 是“无序且元素不重复”的集合，不能索引或切片；非空 set 可用花括号，空 set 应用 `set()`。官方文档同时规定 dict 的 key 必须可哈希，并且现代 Python 的 dict 保留插入顺序。[1][2]

3 知识点清单

1. 集合 `set` 的定义、特点与创建方式。
2. 空集合 `set()` 与空字典 `{}` 的区别。
3. 集合的 `add/remove/pop/clear` 基础操作。
4. 交集、并集、差集及其方法写法和运算符写法。
5. 集合推导式的基本结构。
6. 星号 `*` 在函数调用中的参数拆分；集合次序不保证。
7. 字典 `dict` 的 key-value 映射思想。
8. 字典 key 的唯一性与可哈希要求；value 可为多种类型。
9. 字典的添加、修改、删除与查询。
10. `d[key]` 与 `d.get(key)` 的区别。
11. `keys()`、`values()`、`items()` 的作用。
12. 字符串、列表、元组、集合、字典的最基础差异。
13. `input()` 返回字符串这一事实。
14. `match/case` 的最基础写法，以及为什么购物车菜单应匹配 `"1"`、`"2"` 等字符串。[3][4]
15. 用嵌套字典表示“商品名 -> 价格/数量”的简单购物车。

4 核心概念详解

4.1 集合 set：无序、去重、可修改

基本定义。集合是由互不重复的可哈希元素组成的无序容器。常见用途是成员判断、去重，以及交集、并集、差集等集合运算。[1][2]

通俗理解。可以把集合想成“一个不关心座位号的盒子”：只关心某个元素有没有出现，不关心它排在第几个位置；同样的元素放很多次，最终只保留一份。

创建规则。

```
s1 = {1, 2, 3}     # 非空集合
s2 = set()         # 空集合
```

最容易错： `{}` 不是空集合，而是空字典。判断类型可用 `type(obj)`。

适用范围。集合元素必须可哈希；例如整数、字符串通常可以放入 set，而列表本身不能直接作为 set 元素。

最基础使用步骤。

1. 先确定只关心“是否存在/是否重复”，不关心位置。
2. 用 `{...}` 创建非空集合，或用 `set()` 创建空集合。
3. 添加元素用 `add()`；删除指定元素可用 `remove()`。
4. 两个集合之间需要求公共元素、合并元素、只属于左边的元素时，再使用交集、并集、差集。

极简示例。

```
s = {1, 1, 2, 3}
print(s)        # 只保留 1, 2, 3, 每个元素一份
print(2 in s)   # True
```

常见错误。

- 写 `s[0]`：错误，因为 set 不支持位置索引。
- 期待 `print(s)` 每次都严格按自己写入的顺序显示：不应依赖这一点。
- 把 `{}` 当空集合：错误。

记忆提示： set = 去重 + 不按位置访问 + 可做集合运算

4.2 集合的四个修改方法

方法	作用	初学者注意点
`s.add(x)`	把 x 加入集合	x 已存在时集合仍只保留一份
`s.remove(x)`	删除指定元素 x	x 不存在时会抛出 KeyError
`s.pop()`	删除并返回任意一个元素	不要猜它会删哪一个；空集合调用会出错
`s.clear()`	清空集合	执行后仍是 set，只是长度变成 0

极简示例：

```
s = {1, 2}
s.add(3)       # {1, 2, 3}, 顺序仅作示意
s.remove(2)    # 剩下 1 和 3
s.clear()      # set()
```

4.3 交集、并集、差集

设

$$A = \{1, 2, 3\}, \qquad B = \{2, 3, 4\}.$$

1. 交集 intersection

用途：找两个集合共同拥有的元素。

$$A \cap B = \{2, 3\}.$$

Python 两种基础写法：

```
A.intersection(B)
A & B
```

这里 A、B 是已知集合；结果是一个新集合。

2. 并集 union

用途：把两个集合中的所有不同元素合并起来。

$$A \cup B = \{1, 2, 3, 4\}.$$

```
A.union(B)
A | B
```

3. 差集 difference

用途：保留左边集合中“右边没有”的元素。

$$A \setminus B = \{1\}, \qquad B \setminus A = \{4\}.$$

```
A.difference(B)
A - B
```

差集有方向：A - B 与 B - A 一般不同。做题时先读成一句话：“左边有、右边没有”。

4.4 集合推导式

基本规则：

```
{表达式 for 变量 in 可迭代对象 if 条件}
```

其中：

- “变量”每次接收一个元素；
- “可迭代对象”可以是列表、字符串等；
- “if 条件”可省略；保留时表示只选择满足条件的元素；
- 最外层花括号表示最终得到一个 set。

极简示例：从列表 [1,2,2,3] 中取大于 1 的数并去重：

```python
s = {x for x in [1, 2, 2, 3] if x > 1}
# 结果包含 2 和 3
```

记忆提示： 先 for 逐个拿，再 if 筛，最左边写要放进集合的结果

4.5 星号 *：把可迭代对象拆成多个函数参数

在函数调用中，*iterable 可以把其中的元素依次作为位置参数传入。例如：

```python
a = [10, 20, 30]
print(*a)      # 等价于 print(10, 20, 30)
```

若 a 是 set，也可以写 print(*a)，但因为集合没有受保证的顺序，**不能把输出次序当成知识点或程序逻辑的一部分**。[1][2]

4.6 字典 dict：通过 key 找 value

基本定义。字典是映射类型：把可哈希的 key 映射到任意对象 value。字典本身可修改。[1]

通俗理解。字典像“通讯录”：你不是问“第 0 个联系人是谁”，而是拿“姓名”这个 key 去查对应的“电话号码”这个 value。

基本语法：

```python
score = {
    "小王": 90,
    "小李": 85
}
```

符号解释。

- {}：字典的外层花括号；
- " 小王"：key；
- 冒号 :：把 key 与 value 配成一对；
- 90：value；
- 逗号 ,：分隔不同键值对。

key 的规则。

- key 必须可哈希；字符串、整数通常最常用。
- 同一字典中不能同时保留两个相等的 key；后赋的值会覆盖前面的值。
- value 可以重复，也可以是字符串、数字、列表、字典等多种对象。

最基础示例：

```python
d = {"apple": 5, "banana": 3}
print(d["apple"])   # 5
```

记忆提示： dict = key:value；查 value 要先拿 key

4.7 字典的增、改、删、查

1. 添加与修改：同一条语句

```
d[key] = value
```

如果 key 原来不存在，这是“添加”；如果 key 已存在，这是“修改/覆盖”。

示例：

```
d = {"a": 1}
d["b"] = 2   # 添加
d["a"] = 9   # 修改
```

2. 查询：d[key] 与 d.get(key)

```
d["a"]
d.get("a")
```

两者都能在 key 存在时取得 value。最基础区别是：如果 key 不存在，`d[key]` 会抛出 `KeyError`；`d.get(key)` 默认返回 `None`，也可以提供默认值。[1]

```
d = {"a": 1}
print(d.get("b"))        # None
print(d.get("b", 0))     # 0
```

3. 删除：pop 与 del

```
value = d.pop(key)
del d[key]
```

`pop` 会把被删 key 对应的 value 返回；`del` 只是执行删除语句。

4. 获取所有 key/value/item

```
d.keys()      # 所有 key 的视图
d.values()    # 所有 value 的视图
d.items()     # 所有 (key, value) 对的视图
```

遍历购物车时最常用的是：

```
for name, info in shopping_cart.items():
    ...
```

此时每次循环中，`name` 接收 key，`info` 接收对应 value。

4.8 五类常见容器的最低限度对比

类型	是否按位置有序	可变	可重复	位置索引/切片	最典型用途
str	是	否	字符可重复	支持	文本
list	是	是	是	支持	有序、需要修改的数据
tuple	是	否	是	支持	不希望被修改的一组记录
set	不保证	是	否	不支持	去重、成员判断、集合运算
dict	保留插入顺序	是	key 唯一	不按位置索引	key 到 value 的映射

4.9 input 与 match/case：购物车为什么匹配字符串

Python 官方文档说明，`input()` 读取一行输入，去掉行尾换行后返回一个字符串。[4] 因此：

```
choice = input("请输入 1-5: ")
```

用户键入数字键 1 后，`choice` 的值是字符串 `"1"`，而不是整数 `1`。

最基础的 match/case：

```
match choice:
    case "1":
        print("添加")
    case "2":
        print("修改")
    case _:
        print("其他输入")
```

`match` 后是待匹配的值；每个 `case` 后写一个模式；`case _:` 是最常见的兜底分支。结构化模式匹配自 Python 3.10 引入。[3][5][6]

5 最基础示例

5.1 示例 1：去重

```
nums = [1, 1, 2, 3, 3]
unique_nums = set(nums)
print(unique_nums)
```

思路：列表允许重复，set 不允许重复，因此转换为 set 后相同值只保留一份。不要要求输出顺序与原列表一致。

5.2 示例 2：求共同元素

```
a = {1, 2, 3}
```

```
b = {2, 3, 4}
print(a & b)
```

思路： & 是交集，结果只保留两边都有的 2 和 3。

5.3 示例 3：用字典保存成绩

```
scores = {"小王": 90, "小李": 85}
scores["小张"] = 88
scores["小王"] = 95
print(scores["小王"])
```

思路： 不存在的 key 是添加；已存在的 key 是修改；最后按 key 查询。

5.4 示例 4：最小购物车数据结构

```
shopping_cart = {}
shopping_cart["苹果"] = {
    "商品价格": 5.0,
    "商品数量": 2
}
print(shopping_cart["苹果"]["商品价格"])
```

第一层 key 是商品名“苹果”；第一层 value 又是一个字典，里面再用“商品价格”“商品数量”作为 key。这叫**嵌套字典**，这里只需要理解成“字典里又放了一个字典”。

6 经纠错后的完整购物车基础程序

下面给出与原图需求一致、但语法和逻辑修正后的入门版本。代码只使用当前图片范围内最相关的知识：字典、input()、match/case、if、循环与基础类型转换。

```
shopping_cart = {}

menu = """
##### 购物车系统 #####
1. 添加商品
2. 修改商品
3. 删除商品
4. 查询商品
5. 退出系统
"""

while True:
    print(menu)
    choice = input("请输入您的操作(1-5): ").strip()

    match choice:
        case "1":
```

```
            name = input("请输入所添加的商品名称: ").strip()
            price = float(input("请输入所添加的商品价格: "))
            num = int(input("请输入所添加的商品数量: "))
            shopping_cart[name] = {
                "商品价格": price,
                "商品数量": num
            }
            print("添加成功。")

        case "2":
            name = input("请输入所修改的商品名称: ").strip()
            if name in shopping_cart:
                price = float(input("请输入修改后的商品价格: "))
                num = int(input("请输入修改后的商品数量: "))
                shopping_cart[name] = {
                    "商品价格": price,
                    "商品数量": num
                }
                print("修改成功。")
            else:
                print("无此商品，请重新输入。")

        case "3":
            name = input("请输入所删除的商品名称: ").strip()
            if name in shopping_cart:
                shopping_cart.pop(name)
                print("删除成功。")
            else:
                print("无此商品，请重新输入。")

        case "4":
            if not shopping_cart:
                print("购物车为空。")
            else:
                for name, info in shopping_cart.items():
                    print(
                        f"商品名称:{name} "
                        f"商品价格:{info['商品价格']} "
                        f"商品数量:{info['商品数量']}"
                    )

        case "5":
            print("已退出购物车系统。")
            break

        case _:
            print("请输入 1-5 之间的选项。")
```

逐段理解：

1. `shopping_cart = {}`：这里必须是字典，因为要保存“商品名 -> 商品信息”的映射。
2. `while True`：反复显示菜单，直到用户选择 5。
3. `choice = input(...)`：得到字符串，所以 case 写 `"1"` 到 `"5"`。
4. 添加/修改都使用 `shopping_cart[name] = ...`；区别是修改前先检查 name 是否已存在。
5. 删除用 `pop(name)`。
6. 查询用 `items()` 同时得到商品名和商品信息。
7. 退出用 `break` 结束无限循环。

7 知识点—题目对应表

题号	题型	对应知识点	难度	主要能力
1	单选	空集合创建	入门	语法识别
2	单选	set 不允许重复元素	入门	概念识别
3	单选	add() 添加元素	入门	方法识别
4	单选	并集运算符 |	入门	运算符识别
5	单选	差集有方向	基础	一步判断
6	单选	字典按 key 查 value	入门	语法识别
7	单选	get() 缺失 key 的基本行为	基础	方法区别
8	单选	dict key 必须可哈希	入门	类型判断
9	单选	items() 返回键值对视图	入门	方法识别
10	单选	input() 返回 str，match 匹配字符串	基础	语法判断
11	多选	集合交并差	入门	逐项识别
12	多选	set 的性质和操作	入门	概念判断
13	多选	dict 增删改查	入门	方法判断
14	多选	可作字典 key 的对象	基础	类型判断
15	多选	购物车基本程序逻辑	基础	代码阅读
16	填空	并集运算符	入门	语法记忆
17	填空	差集表达式	入门	语法记忆
18	填空	字典按 key 查询	入门	语法记忆
19	填空	items()	入门	方法记忆
20	填空	空 set 写法	入门	语法记忆
21	解答	交集、并集、差集	基础	直接运算
22	解答	set 添加和删除	基础	步骤模仿
23	解答	dict 添加、修改、查询	基础	步骤模仿
24	解答	input + match + 嵌套字典	基础	轻度综合

8 单项选择题（10 道）

说明：每题 4 个选项，只有一个正确答案。

1. 在 Python 中，创建**空集合**的正确写法是（　）。

A. `{}`

B. `set()`

C. `[]`

D. `()`

2. 关于集合 set，下面最符合其基本特点的是（　）。

A. 元素允许重复，并且一定按位置编号

B. 元素不重复，且不支持普通位置索引

C. 元素不重复，但集合永远不可修改

D. 集合只能存字符串

3. 若 `s = {1, 2}`，想把元素 3 添加到 s 中，应使用（　）。

A. `s.add(3)`

B. `s.append(3)`

C. `s.push(3)`

D. `s.insert(3)`

4. 两个集合 `a` 和 `b` 的并集可以写成（　）。

A. `a & b`

B. `a - b`

C. `a | b`

D. `a + b`

5. 已知 `a = {1,2,3}`，`b = {2,3,4}`，那么 `a - b` 的结果包含（　）。

A. 只有 1

B. 只有 4

C. 2 和 3

D. 1、2、3、4

6. 已知 `d = {"name": "Tom"}`，要取得字符串 `"Tom"`，最直接的写法是（　）。

A. `d[0]`

B. `d["name"]`

C. `d.name`

D. `d(0)`

7. 若 `d = {"a": 1}`，执行 `d.get("b")`，在没有提供默认值时最基本的结果是（　）。

A. 一定返回 0

B. 返回 `None`

C. 返回字符串 `"b"`

D. 自动创建键 `"b"`

8. 下面哪一个对象**不能**直接作为普通字典的 key（　）。

A. 字符串 `"apple"`

B. 整数 `10`

C. 元组 `(1,2)`

D. 列表 `[1,2]`

9. 若想同时遍历字典的 key 和 value，最合适的基础方法是（　）。

A. `d.keys()`

B. `d.values()`

C. `d.items()`

D. `d.clear()`

10. 有代码 `choice = input(" 请输入： ")`。如果用户键入 1，下面哪个 case 最直接匹配这个输入（　）。

A. `case 1:`

B. `case "1":`

C. `Case1:`

D. `case = 1:`

9 多项选择题（5 道）

评分说明：每题有 2 个或 3 个正确选项；全部选对得分，选错不得分。

11. 关于集合 `a`、`b`，下列对应关系正确的有（　）。

A. `a & b` 表示交集

B. `a | b` 表示并集

C. `a - b` 表示“a 中有但 b 中没有”的元素

D. `a + b` 表示并集

12. 关于 Python 的 set，下列说法正确的有（　）。

A. 空集合可用 `set()` 创建

B. set 不支持像列表那样用 `s[0]` 取第一个元素

C. `add()` 可添加元素

D. set 中可以同时保留两个完全相同的元素

13. 设 `d = {"a":1}`，下列操作或说法正确的有（　）。

A. `d["b"] = 2` 可以添加新键值对

B. `d["a"] = 9` 可以修改已有 key 对应的 value

C. `d.pop("a")` 可以删除 key 为 `"a"` 的键值对并返回其 value

D. `d[0]` 一定等于字典中“第一个 value”

14. 下列对象中，通常可以直接作为普通字典 key 的有（　）。

A. `"name"`

B. `100`

C. `(1, 2)`

D. `[1, 2]`

15. 关于原图购物车程序的规范写法，下列说法正确的有（　）。

A. 若 `choice` 来自 `input()`，可写 `case "1":`

B. 修改商品时应先输入要修改的商品名称，再判断该名称是否在购物车中

C. 商品数量若希望作为整数使用，可以写 `int(input(...))`

D. Python 中正确分支关键字写作 `Case1:`

10 填空题（5 道）

16. 集合 `a` 与 `b` 的并集，使用运算符可写成 _____。

17. 集合 `a` 中有、但 `b` 中没有的元素，使用差集运算符可写成 _____。

18. 已知 `d = {"age": 18}`，按 key 直接取得 18 的写法是 _____。

19. 若要获得字典 `d` 的所有 key-value 对的视图，应调用 _____。

20. Python 中创建空集合的标准写法是 _____。

11 基础解答题（4 道）

21. 已知：

```
a = {1, 2, 3}
b = {2, 3, 4}
```

分别写出：`a & b`、`a | b`、`a - b` 的结果，并用一句话说明三种运算各自表示什么。

22. 已知：

```
s = {1, 2}
```

按顺序完成两步：先把 3 加入集合，再把 1 删除。写出两行 Python 代码，并写出操作结束后集合包含哪些元素。

23. 已知字典：

```
scores = {"小王": 80}
```

完成下列三步：

(1) 添加“小李”，成绩 90；

(2) 把“小王”的成绩改为 85；

(3) 查询“小李”的成绩。

写出最基础的 Python 代码，并说明为什么“添加”和“修改”可以使用同一种 `d[key] = value` 写法。

24. 补全下面的最小菜单分支，使得用户输入 1 时向购物车加入“苹果”，价格为 5.0，数量为 2；用户输入其他内容时只打印“其他操作”。

```
shopping_cart = {}
choice = input("请输入操作: ")

match choice:
    __________:
        shopping_cart["苹果"] = {
            "商品价格": 5.0,
            "商品数量": 2
        }
    __________:
        print("其他操作")
```

写出两个空格处的完整代码，并解释为什么第一个 case 不是 `case 1:`。

12 答案速查表

题号	答案	题号	答案
1	B	11	ABC
2	B	12	ABC
3	A	13	ABC
4	C	14	ABC
5	A	15	ABC
6	B	16	`a | b`
7	B	17	`a - b`
8	D	18	`d["age"]`
9	C	19	`d.items()`
10	B	20	`set()`

解答题 21–24 的标准过程见后文详细解析。

13 单项选择题详细解析

第 1 题

正确答案：B。

考查知识点：空集合的创建。

相关规则：非空集合可以用花括号写，例如 `{1,2}`；但空花括号 `{}` 被 Python 定义为空字典，因此空集合要写 `set()`。[1][2]

逐项判断：

- A：**错误**。`{}` 是空 dict，不是空 set。
- B：**正确**。`set()` 返回一个空集合。
- C：**错误**。`[]` 是空列表。
- D：**错误**。`()` 是空元组。

常见误区：看到数学集合常用花括号，就以为 Python 的空集合也写 `{}`。Python 在这里有语法歧义，因此把 `{}` 留给了 dict。

一句总结： 空 set = set()；空 dict = {}

第 2 题

正确答案：B。

考查知识点：set 的核心性质。

定义：set 是无序的、元素互不重复的集合类型；它不记录元素位置，因此不支持像列表那样的索引和切片。[1]

逐项判断：

- A：**错误**。set 不保证位置顺序，也不保留重复元素。
- B：**正确**。正好描述了“不重复 + 不支持普通位置索引”。
- C：**错误**。普通 `set` 是可变的，可以 `add/remove`；不可变的是 `frozenset`。
- D：**错误**。set 可以存许多可哈希对象，不只字符串。

常见误区：把 set 与 list 混淆。list 强调“位置与顺序”，set 强调“是否存在与是否重复”。

一句总结：set 不靠下标认元素，而靠“成员是否存在”认元素。

第 3 题

正确答案：A。

考查知识点：`add()`。

规则：`s.add(x)` 把元素 x 加入 set s。若 x 已经存在，集合仍只保留一份。

逐项判断：

- A：**正确**。set 的标准添加方法。
- B：**错误**。`append()` 是列表常用方法，不是 set 方法。
- C：**错误**。普通 set 没有 `push()` 方法。
- D：**错误**。普通 set 没有列表式 `insert()` 位置插入，因为 set 没有位置索引。

一句总结：list 常用 append；set 常用 add。

第 4 题

正确答案：C。

考查知识点：集合运算符。

规则：

`a & b` = 交集，　`a | b` = 并集，　`a - b` = 差集.

逐项判断：

- A：**错误**，`&` 是交集。
- B：**错误**，`-` 是差集。
- C：**正确**，`|` 是并集。
- D：**错误**，两个 set 不能用普通 `+` 表示并集。

一句总结：交 &，并 |，差 -。

第 5 题

正确答案：A。

考查知识点：差集方向。

公式/规则：`a - b` 的意思是“从 a 中保留那些不在 b 中的元素”。本题

$$a = \{1, 2, 3\}, \quad b = \{2, 3, 4\}.$$

逐个看 a：1 不在 b 中，保留；2 在 b 中，去掉；3 在 b 中，去掉。所以

$$a - b = \{1\}.$$

逐项判断：A 正确；B 是 `b-a` 的结果；C 是交集；D 是并集。

常见误区：把差集当成普通数字相减，或者忘记左右顺序。

一句总结：a-b：只看 a，删掉 b 里也有的。

第 6 题

正确答案：B。

考查知识点：字典按 key 查询。

规则：`d[key]` 返回 key 对应的 value。字典不是按“第几个元素”的位置索引来取值。

逐项判断：

- A：**错误**。`0` 只有在它本身就是字典 key 时才有意义，不代表“第 0 项”。
- B：**正确**。key 正是 `"name"`，得到 `"Tom"`。
- C：**错误**。普通 dict 不用属性点号查询普通 key。
- D：**错误**。字典对象不是这样用圆括号调用。

一句总结：dict 不是“第几个”，而是“哪个 key”。

第 7 题

正确答案：B。

考查知识点：`get()` 的基础行为。

规则：当 key 存在时，`d.get(key)` 返回对应 value；当 key 不存在且未给默认值时，返回 `None`。这与 `d[key]` 的缺失行为不同，后者会抛出 `KeyError`。[1]

逐项判断：A、C、D 都不是默认行为；B 正确。

一句总结：get 找不到时，默认给 None，而不是自动造一个键。

第 8 题

正确答案：D。

考查知识点：字典 key 的可哈希要求。

规则：普通 dict 的 key 必须是可哈希对象。字符串、整数，以及只包含可哈希元素的元组通常可作 key；列表可修改且不可哈希，不能直接作为普通 dict key。[1]

逐项判断：A、B、C 可以；D 不可以。

常见误区：把“key 必须不可变”机械化。更精确的关键词是“可哈希”。入门阶段先记住：字符串、数字最安全，list 不行。

一句总结：字典 key 最常用字符串；list 不能直接当 key。

第 9 题

正确答案：C。

考查知识点：`keys/values/items`。

规则：

- `d.keys()`：看所有 key；
- `d.values()`：看所有 value；
- `d.items()`：看每一对 key-value。

所以需要“同时遍历 key 与 value”时最适合 `items()`。

逐项判断：A 只有 key；B 只有 value；C 两者成对；D 会清空字典。

一句总结：items = 一项一项的 (key, value)。

第 10 题

正确答案：B。

考查知识点：`input()` 返回字符串；`match/case` 字面量匹配。

规则说明：官方文档规定 `input()` 读取一行并返回字符串。[4] 用户键入 1 后，变量值是 `"1"`。`match` 会拿 subject 与 case 模式比较；字符串文字应写带引号的 `"1"`。[3][5]

逐项判断：

- A：**错误**。整数 1 与字符串 `"1"` 不是同一个值。
- B：**正确**。直接匹配输入得到的字符串。
- C：**错误**。Python 关键字小写，而且需要 `case "1":` 这种语法。
- D：**错误**。case 后不是赋值语句。

一句总结：input 先记成“永远先得到 str”。

14 多项选择题详细解析

第 11 题

正确答案：ABC。

考查知识点：交集、并集、差集的运算符。

- A：**正确**。`a & b` 返回两者共有元素，即交集。
- B：**正确**。`a | b` 合并两者所有不同元素，即并集。
- C：**正确**。`a - b` 只保留 a 中不在 b 中的元素。
- D：**错误**。普通 set 不用 `+` 求并集。

总结： ABC；交 &，并 |，差 -。

第 12 题

正确答案：ABC。

考查知识点： set 的创建、索引与添加。

- A：**正确**。空 set 写 `set()`。
- B：**正确**。set 无位置索引，不支持 `s[0]` 这种列表式访问。
- C：**正确**。`add()` 是 set 的基础添加方法。
- D：**错误**。set 的核心性质之一是元素不重复。

总结： 空 set、无索引、用 add、自动去重。

第 13 题

正确答案：ABC。

考查知识点： dict 的增、改、删。

- A：**正确**。`"b"` 原来不存在，所以是添加。
- B：**正确**。`"a"` 已存在，所以同一语句变成更新。
- C：**正确**。`pop("a")` 删除该键并返回其 value。
- D：**错误**。`d[0]` 是“拿 key 0 去查”，不是“取第一个 value”。如果 key 0 不存在，会出错。

总结： dict 的方括号里写 key，不写位置序号。

第 14 题

正确答案：ABC。

考查知识点： 可哈希 key。

- A：**正确**。字符串可哈希。
- B：**正确**。整数可哈希。
- C：**正确**。该元组只含整数，因此可哈希。
- D：**错误**。列表可变、不可哈希，不能直接作为普通 dict key。

总结： 最常见可用 key：str、int、合适的 tuple；list 不行。

第 15 题

正确答案：ABC。

考查知识点：购物车中 input、match、变量顺序与类型转换。

- A：**正确**。`input()` 返回 str，所以菜单 1 可匹配 `case "1":`。
- B：**正确**。先得到要修改的 name，才能判断 `name in shopping_cart`。
- C：**正确**。`int(input(...))` 把输入字符串转换成整数数量。
- D：**错误**。Python 关键字是小写 `case`，且必须写模式，例如 `case "1":`。

总结： 先输入，再判断；input 是 str；case 小写。

15 填空题详细解析

第 16 题

标准答案：`a | b`。

使用规则：竖线 `|` 在两个 set 之间表示并集，等价于 `a.union(b)`。

推理步骤：题目关键词是“并集”→ 找所有不同元素的合并 → 运算符写 `|`。

格式要求：左右两边必须是 set；写 `a.union(b)` 也表达同一集合运算，但本题明确要求“使用运算符”。

易错点：把 `|` 与 `&` 记反。

第 17 题

标准答案：`a - b`。

使用规则：差集 `a - b` 表示“a 中有、b 中没有”。

推理步骤：题干已经给出方向“a 中有但 b 中没有”，所以左边写 a，右边写 b，中间写减号。

易错点：写成 `b - a` 会把方向反过来。

第 18 题

标准答案：`d["age"]`。

使用规则：`d[key]` 用 key 直接取得 value。

代入：本题 key 是字符串 `"age"`，所以放入方括号：`d["age"]`，结果为 18。

可接受等价答案：`d.get("age")` 也能得到 18，但题目问“按 key 直接取得”的最典型写法，因此标准答案为方括号形式。

第 19 题

标准答案：`d.items()`。

使用规则： `items()` 提供字典的 key-value 对视图；`keys()` 只有 key，`values()` 只有 value。

易错点： 把 items 误记成“所有 value”。记忆英文：item 可理解为“一项”，一项字典内容就是一对 key-value。

第 20 题

标准答案： `set()`。

使用规则： 空集合必须用 set 构造器；`{}` 是空字典。

易错点： 这是本章最常见的纯语法错误，应直接记牢。

16 解答题详细解析

第 21 题：交集、并集、差集

题意分析： 已知两个非常简单的集合 a、b，要求分别做三种集合运算。

知识定位： 使用 set 的交集 `&`、并集 `|`、差集 `-`。

规则说明：

- `a & b`：只保留两边都有的元素；
- `a | b`：保留两边出现过的所有不同元素；
- `a - b`：只保留 a 中有、b 中没有的元素。

本题中 a、b 都是 set，因此可以直接使用这三个运算符。

解题步骤：

1. 看共同元素：2、3 同时出现在 a 和 b 中，所以

$$\texttt{a \& b} = \{2, 3\}.$$

2. 合并所有不同元素：1、2、3、4，所以

$$\texttt{a | b} = \{1, 2, 3, 4\}.$$

3. 从 a 的 1、2、3 中删除 b 也有的 2、3，只剩 1，所以

$$\texttt{a - b} = \{1\}.$$

规范答案：

```
a & b   # {2, 3}
a | b   # {1, 2, 3, 4}
a - b   # {1}
```

集合打印顺序不应作为评分点，只要元素集合正确即可。

结果检查： 交集一定只能来自两边共同部分；并集的元素数不会少于任一集合；`a-b` 的结果一定是 a 的子集。本题结果符合这三个检查。

易错提醒：不要把 a-b 写成“元素逐项数字相减”。

方法总结：交集找共同，并集合并去重，差集看“左有右无”。

第 22 题：set 的添加与删除

题意分析：初始 set 为 {1,2}；先添加 3，再删除 1。

知识定位：add() 和 remove()。

方法说明：

- s.add(x)：把 x 加入 s；
- s.remove(x)：删除指定 x。

本题明确知道要删除的是 1，所以用 remove(1) 比 pop() 更合适，因为 pop() 不保证删哪个元素。

解题步骤：

```
s.add(3)
s.remove(1)
```

第一步后 s 含 1、2、3；第二步删去 1，因此最后只含 2、3。

规范答案：代码如上，最终集合包含元素 2 和 3。

结果检查：题目只添加一个新元素、删除一个已有元素，所以最终仍有两个元素；2 没被操作，3 被加入，1 被删除，结论一致。

易错提醒：不要写 s.append(3)；那是 list 的典型方法。

方法总结：set 指定添加用 add，指定删除用 remove。

第 23 题：dict 的添加、修改与查询

题意分析：初始字典只有“小王”：80。要求添加“小李”：90，修改“小王”为 85，再查询“小李”。

知识定位：d[key] = value 与 d[key]。

规则说明：d[key] = value 的含义取决于 key 是否已存在：不存在时创建新键值对，存在时覆盖旧 value。因此“添加”和“修改”不需要两种完全不同的语法。

解题步骤：

```
scores["小李"] = 90
scores["小王"] = 85
print(scores["小李"])
```

逐步解释：

1. " 小李" 原来不在 scores 中，所以第一行是添加。
2. " 小王" 原来已经存在，所以第二行把 80 覆盖成 85。
3. 第三行拿 key " 小李" 查询，得到 90。

结果整理：最后字典中“小王”为 85，“小李”为 90；打印结果为 90。

结果检查： key 不重复，因此“小王”只保留一个条目，value 应是最后一次赋的 85。

易错提醒： 不要把 `scores[0]` 当成第一个成绩；方括号里是 key。

方法总结： 先看 key 在不在：不在就是加，在就是改；查值仍用同一个 key。

第 24 题：input + match + 嵌套字典

题意分析： 用户输入菜单选项；输入 1 时添加固定商品“苹果”，其他输入进入兜底分支。

知识定位： `input()`、`match/case`、字典赋值。

规则说明：

- `input()` 返回字符串，所以用户键入 1 得到 `"1"`。[4]
- `match choice:` 后逐个匹配 case；字符串字面量要带引号。[3][5]
- `case _:` 是兜底模式，可匹配前面没有处理的其他输入。

补全代码：

```
shopping_cart = {}
choice = input("请输入操作: ")

match choice:
    case "1":
        shopping_cart["苹果"] = {
            "商品价格": 5.0,
            "商品数量": 2
        }
    case _:
        print("其他操作")
```

为什么不是 case 1： 因为没有做 `int(input(...))` 转换，`choice` 仍是 str。字符串 `"1"` 与整数 `1` 类型和值都不同，直接写 `case 1:` 不会匹配用户输入得到的 `"1"`。

结果检查： 输入 1 后，第一层字典增加 key“苹果”；对应 value 是一个包含“商品价格”和“商品数量”两个 key 的字典。输入其他字符则不修改购物车，只打印提示。

易错提醒： 一是忘记 case 后的冒号；二是写大写 `Case`；三是忘记字符串引号。

方法总结： 先确认输入类型，再写 case；输入没转 int，就按字符串匹配。

17 高频易错点总结

1. **空集合：** `set()`，不是 `{}`。
2. **集合不能位置索引：** 不要写 `s[0]`。
3. **集合输出顺序：** 不要依赖打印/迭代的具体次序。
4. **差集方向：** `a-b` 是“a 有、b 没有”。
5. **set 添加：** 用 `add()`，不是 list 的 `append()`。

6. **字典访问：**方括号里是 key，不是“第几个”。
7. **重复 key：**后写的 value 覆盖前面的 value。
8. **字典 key：**必须可哈希；list 不能直接作 key。
9. **get 与方括号：**缺失 key 时，`get()` 默认给 `None`；`d[key]` 会抛 `KeyError`。
10. **input 类型：**`input()` 返回 str；需要数字就显式 `int(...)` 或 `float(...)`。
11. **match/case：**关键字小写；`case` 后是模式；输入字符串要匹配字符串。
12. **修改购物车：**先输入商品名，再判断是否存在，再输入新价格/数量并真正写回字典。

18 基础解题步骤总结

遇到集合题

1. 先看是“添加/删除”还是“交并差”。
2. 添加指定元素：`add`；删除指定元素：`remove`。
3. 共同元素：`&`；所有不同元素：`|`；左有右无：`-`。
4. 最后检查：集合结果不要含重复元素，不要用位置顺序作为依据。

遇到字典题

1. 先找清楚谁是 key，谁是 value。
2. 查 value：`d[key]` 或 `d.get(key)`。
3. 新增/修改：`d[key] = value`。
4. 删除：基础题常用 `pop(key)` 或 `del d[key]`。
5. 同时遍历 key 和 value：`items()`。

遇到菜单 + match 题

1. 先看选择变量从哪里来。
2. 若直接来自 `input()`，先按 str 理解。
3. 因此用户输入 1 常写 `case "1":`。
4. 最后加 `case _:` 处理其他输入；需要退出循环时使用 `break`。

19 参考资料与核验来源

本资料的技术事实优先使用 Python Software Foundation 的官方文档与正式 PEP。由于系统性技术问题应以一手规范为准，本资料没有把个人博客、聚合题库或问答网站作为核心依据。以下链接访问日期均为 2026-08-19。

[1] Python Software Foundation. *Python 3.13.15 Documentation – Built-in Types.* 重点核验 set 的无序/不重复/索引限制、交并差运算、dict 的映射性质、key 的可哈希要求、dict 插入顺序、常用字典操作。
https://docs.python.org/3.13/library/stdtypes.html

[2] Python Software Foundation. *Python 3.13.15 Tutorial – Data Structures.* 重点核验空 set 必须写 `set()`、集合运算示例、集合推导式、字典入门说明。
https://docs.python.org/3.13/tutorial/datastructures.html

[3] Python Software Foundation. *Python 3.13.15 Language Reference – Compound statements, The match statement.* 重点核验 `match/case` 的语法、匹配流程与 `case _:` 所属的模式匹配语义。
https://docs.python.org/3.13/reference/compound_stmts.html

[4] Python Software Foundation. *Python 3.13.15 Documentation – Built-in Functions, input().* 重点核验 `input()` 读取并返回字符串。
https://docs.python.org/3.13/library/functions.html#input

[5] Brandt Bucher, Guido van Rossum. *PEP 634 – Structural Pattern Matching: Specification.* Python Enhancement Proposals, Python 3.10. 重点核验结构化模式匹配的正式语法与 literal pattern。
https://peps.python.org/pep-0634/

[6] Tobias Kohn et al. *PEP 636 – Structural Pattern Matching: Tutorial.* Python Enhancement Proposals. 用于对 `match/case` 的入门教学表达进行交叉核验。
https://peps.python.org/pep-0636/

20 质量检查记录

- **图片覆盖：** 5 张图片的主要标题、定义、操作表、容器对比、购物车需求与可见代码均已重新编码；不把姓名等非考点的模糊笔迹用于命题。
- **纠错：** 已明确区分原图写法与修正，重点修复 `Case1/Case2`、变量顺序、变量名覆盖、`input()` 类型、修改未回写等问题。
- **题量：** 单项选择 10 道，多项选择 5 道，填空 5 道，基础解答 4 道，共 24 道。
- **难度：** 全部标记为“入门”或“基础”；不含竞赛、证明、复杂参数讨论或长计算。
- **答案：** 所有题目均有答案；单选唯一，多选每题 3 个正确项；详细解析逐项说明正确与错误原因。
- **代码：** 购物车完整示例按 Python 3.13 语法整理，并对核心语法进行独立语法检查。
- **来源：** 仅使用 Python 官方文档和正式 PEP 作为关键技术核验来源，未虚构参考文献。
- **排版：** 采用 A4、XeLaTeX、ctexart；代码使用 listings；未嵌入低清原图；表格宽度按正文范围设计。